

Normal-3 Dijkstra Sequence

Fundamentals of Data Structures —Laboratory Project Report

Anonymous (for peer review submission)

2026-05-04

Normal-3 Dijkstra Sequence

Chapter 1: Introduction

Dijkstra 算法用于求解带非负权图上的单源最短路径问题。算法维护一个已确定最短路径的顶点集合，每一步都从未确定集合中选择当前距离源点最小的顶点加入集合，并进行松弛操作。

本题要求判断给定顶点排列是否可能作为某一次 Dijkstra 算法的选点序列。由于可能存在多个顶点具有相同的最小暂定距离，因此合法序列不唯一。判定的关键是：在每一步，序列当前顶点是否属于“当前未访问顶点中最小距离”的集合。

题目约束如下：

- 顶点数 $N_v \leq 1000$
- 边数 $N_e \leq 100000$
- 查询数 $K \leq 100$
- 边权为正整数
- 图保证连通

Chapter 2: Algorithm Specification

2.1 Main Data Structures

- 邻接表
 - `edges[]`: 保存边信息 (to, w, next)
 - `head[]`: 每个顶点邻接链表头下标
- 工作数组
 - `distv[]`: 当前最短估计距离
 - `visited[]`: 顶点是否已被选入最短路径集合
 - `seq[]`: 当前查询序列

2.2 Algorithm Idea

对每个查询序列，执行一次“强制按序列取点”的 Dijkstra 模拟：

1. 将序列首元素作为源点 `source`。
2. 初始化 `distv[source]=0`，其余为 `INF`；`visited[]` 全部置 0。
3. 对序列中的每个顶点 `u`:
 - 在线性扫描中求所有未访问顶点的最小距离 `min_dist`；
 - 若 `distv[u] != min_dist`，则该序列不满足 Dijkstra 选点规则，判定 No；
 - 否则将 `u` 标记为 `visited`，并用 `u` 对其邻边做松弛。
4. 若所有步骤均通过，判定 Yes。

2.3 Pseudocode

```
CHECK_SEQUENCE(seq, n):
    source <- seq[0]
    for v in [1..n]:
        dist[v] <- INF
        visited[v] <- false
    dist[source] <- 0

    for i in [0..n-1]:
        u <- seq[i]

        if visited[u] == true:
            return No

        min_dist <- INF
        for v in [1..n]:
            if visited[v] == false:
                min_dist <- min(min_dist, dist[v])
```

```

    if dist[u] != min_dist:
        return No

    visited[u] <- true

    for each edge (u, to, w):
        if visited[to] == false and dist[u] + w < dist[to]:
            dist[to] <- dist[u] + w

    return Yes

```

2.4 Correctness Sketch

- 必要性: Dijkstra 每一步只能选择当前未访问顶点中 dist 最小的顶点之一。若给定序列某步选择的顶点不满足该条件, 则不可能由 Dijkstra 产生。
- 充分性: 若每一步都满足“当前顶点 dist 等于未访问顶点最小 dist ”, 则该序列可视为 Dijkstra 在并列最小值时的一种合法 tie-break 结果。

因此上述判定条件是充要的。

2.5 Complexity

设顶点数为 N , 边数为 M :

- 每一步线性扫描求最小 dist , 复杂度 $O(N)$, 共 N 步, 得到 $O(N^2)$
- 邻接表总松弛复杂度 $O(M)$
- 单个查询总复杂度 $O(N^2 + M)$
- K 个查询总复杂度 $O(K*(N^2 + M))$
- 额外空间复杂度 $O(N + M)$

Chapter 3: Testing Results

3.1 Test Case Table

ID	输入文件	测试目的	期望结果	实际结果	若失败的可能原因	当前状态
T1	code/tests/sample1.in	验证官方样例	Yes, Yes, Yes, No	Yes, Yes, Yes, No	无	pass
T2	code/tests/additional1.in	验证“错误提前选点”会判 No; 验证更换源点的正确性	Yes, No, Yes	Yes, No, Yes	无	pass
T3	code/tests/additional2.in	验证并列最短路时, 多序列可同时合法	Yes, Yes, No	Yes, Yes, No	无	pass
T4	code/tests/additional3.in	验证后续松弛更新导致的顺序差异 (先取非最小应判 No)	Yes, No, Yes	Yes, No, Yes	无	pass

3.2 Raw Outputs

T1 output:

```

Yes
Yes
Yes
No

```

T2 output:

Yes
No
Yes

T3 output:

Yes
Yes
No

T4 output:

Yes
No
Yes

Chapter 4: Analysis and Comments

1. 本实现使用邻接表 + 线性扫描最小 dist，避免了优先队列实现复杂度，逻辑直接且便于验证序列合法性。
2. 在 $N \leq 1000$ 的约束下， $O(N^2)$ 的扫描成本可接受；M 最大 $1e5$ 时，邻接表松弛仍然高效。
3. 该判定方法本质是对 Dijkstra 贪心选点约束的逐步一致性校验，具有清晰的可解释性。
4. 若将来扩展到更大 N，可改为堆优化版本以降低常规最短路开销；但“序列合法性判定”仍需保留每步最小性检查。

Appendix: Source Code (in C)

File: code/src/main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXV 1005
#define MAXE 200005
#define INF 0x3f3f3f3f

/* 邻接表边结点：终点、权重、下一条边下标。 */
typedef struct {
    int to;
    int w;
    int next;
} Edge;

/* 图存储：无向图用两条有向边。 */
static Edge edges[MAXE];
static int head[MAXV];
static int edge_count = 0;

/* Dijkstra 模拟过程中的工作数组。 */
static int distv[MAXV];
static int visited[MAXV];
static int seq[MAXV];

/* 向邻接表头插入一条有向边 u -> v。 */
static void add_edge(int u, int v, int w) {
    edges[edge_count].to = v;
    edges[edge_count].w = w;
    edges[edge_count].next = head[u];
    head[u] = edge_count++;
}

/*
```

```

* 检查给定排列是否可能是某次 Dijkstra 的选点顺序。
* 判定要点：每一步被选中的顶点，必须是当前未收录顶点中 dist 最小者之一。
*/
static int is_dijkstra_sequence(int n) {
    int i;
    int source = seq[0];

    /* 每个查询都要重新初始化访问标记和距离数组。 */
    memset(visited, 0, sizeof(visited));
    for (i = 1; i <= n; ++i) {
        distv[i] = INF;
    }
    distv[source] = 0;

    /* 按给定序列强制选点。 */
    for (i = 0; i < n; ++i) {
        int u = seq[i];
        int v;
        int min_dist = INF;
        int e;

        /* 出现重复顶点，直接非法。 */
        if (visited[u]) {
            return 0;
        }

        /* 找当前所有未访问顶点中的最小 dist。 */
        for (v = 1; v <= n; ++v) {
            if (!visited[v] && distv[v] < min_dist) {
                min_dist = distv[v];
            }
        }

        /* 本步顶点必须满足最小性。 */
        if (distv[u] != min_dist) {
            return 0;
        }

        visited[u] = 1;

        /* 标准松弛操作。 */
        for (e = head[u]; e != -1; e = edges[e].next) {
            int to = edges[e].to;
            int w = edges[e].w;
            if (!visited[to] && distv[u] + w < distv[to]) {
                distv[to] = distv[u] + w;
            }
        }
    }

    return 1;
}

int main(void) {
    int n, m;
    int i;
    int k;

```

```

/* 读取顶点数和边数；输入不完整时直接结束。 */
if (scanf("%d %d", &n, &m) != 2) {
    return 0;
}

/* 邻接表头初始化为 -1, 表示空链。 */
memset(head, -1, sizeof(head));

/* 读入无向图：每条边拆成两条有向边。 */
for (i = 0; i < m; ++i) {
    int u, v, w;
    scanf("%d %d %d", &u, &v, &w);
    add_edge(u, v, w);
    add_edge(v, u, w);
}

/* 逐个查询：每行一个顶点排列。 */
scanf("%d", &k);
while (k--) {
    for (i = 0; i < n; ++i) {
        scanf("%d", &seq[i]);
    }

    /* 输出是否为合法 Dijkstra 选点序列。 */
    if (is_dijkstra_sequence(n)) {
        puts("Yes");
    } else {
        puts("No");
    }
}

return 0;
}

```

Declaration

I hereby declare that all the work done in this project titled “Normal-3 Dijkstra Sequence” is of my independent effort.